AXI4_Lite

FND Controller

I AI 시스템 반도체 설계 2기

I 엄 찬 하

대한상공회의소

guichanha00@gmail.com

MIRI COMPANY

목차 페이지

01

프로젝트 개요

CHAPTER 01

프로젝트 개요

목표

1. AXI BUS 구조 설계 및 이해
2. IP 설계
3. C언어를 통한 시스템 기능 구현

CHAPTER 01

프로젝트 개요

개발 환경

VIVADO

Tool

VIVADO 2020.2

FPGA

Basys3

VIVADO

XILINX VITIS

Design Tool

SystemVerilog

Language

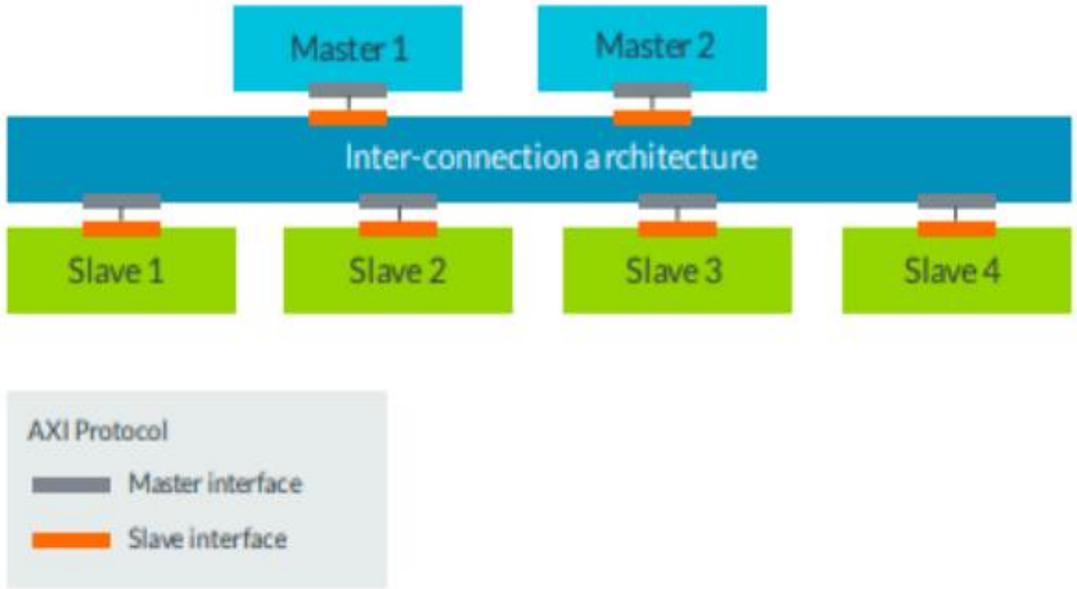
System Verilog / C 언어

02

AXI4_Lite

CHAPTER 02

AXI4_Lite



AXI 란?

ARM에서 정의한 AMBA 인터페이스 사양 중 하나로,
고속 데이터 통신을 위한 인터페이스

AXI 의 포인트 투 포인트 구조

- 마스터 1개와 슬레이브 1개 간 직접 연결
- 다중 마스터, 다중 슬레이브가 동시에 통신 가능
  => 병렬성 극대화
- Broadcasting X
  (하나의 마스터가 동시에 여러 슬레이브에 정보 전달 X)

CHAPTER 02

AXI4_Lite

5개의 독립 채널 구조

=〉 독립적인 채널 기반 구조로 충돌 없이 동시 처리 가능

- AW (Write Address)
- W (Write Data)
- B (Write Response)
- AR (Read Address)
- R (Read Data)

- Write , Read 병렬 처리 가능

Write , Read 트랜잭션이 각각의 채널에서 동시 진행되어 전체 처리 효율을 극대화

⇒ Valid , Ready를 활용한 Hand-Shaking으로 흐름 제어 기능 확보

CHAPTER 02

AXI4_Lite

채널 특성

→ 각 채널은 단방향(Unidirectional) 으로 동작

→ Write Response 채널이 별도로 필요

→ Read는 별도의 응답 채널 없이 Read Data 채널에 응답 포함

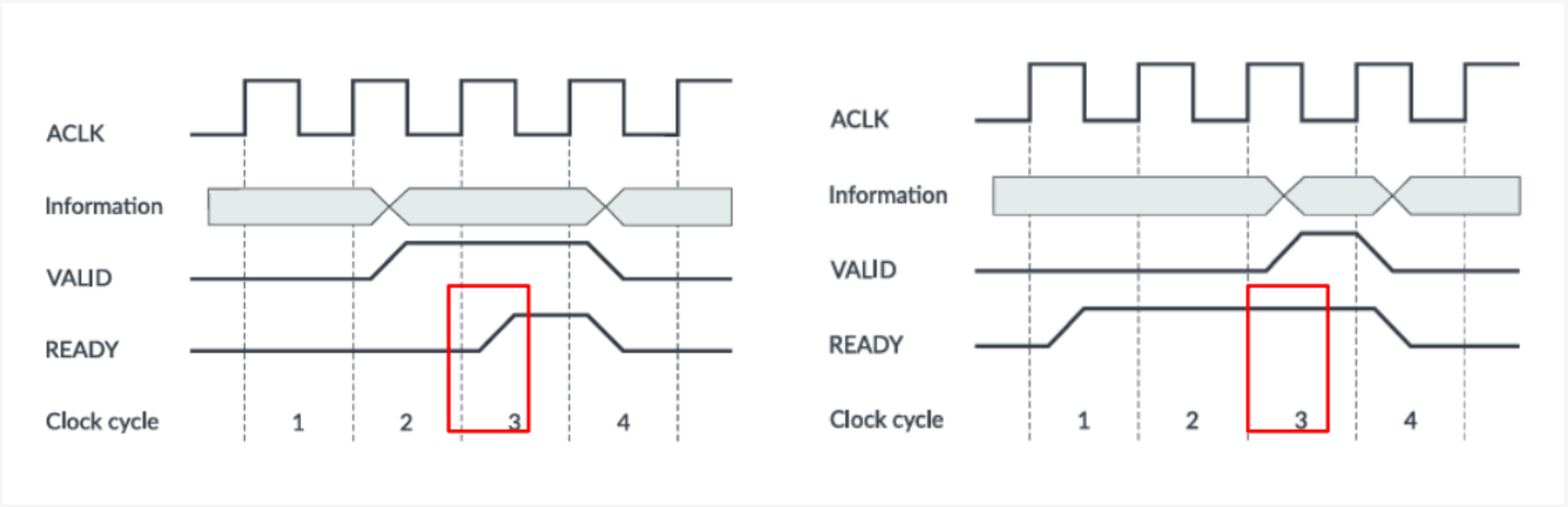
→ 읽기와 쓰기 모두 주소 채널과 데이터 채널을 분리하여 사용

→ 인터페이스의 대역폭을 극대화할 수 있음

→ 읽기 채널과 쓰기 채널 그룹 간에는 타이밍 의존성이 없어 읽기 쓰기 동시에 진행

CHAPTER 02

AXI4_Lite

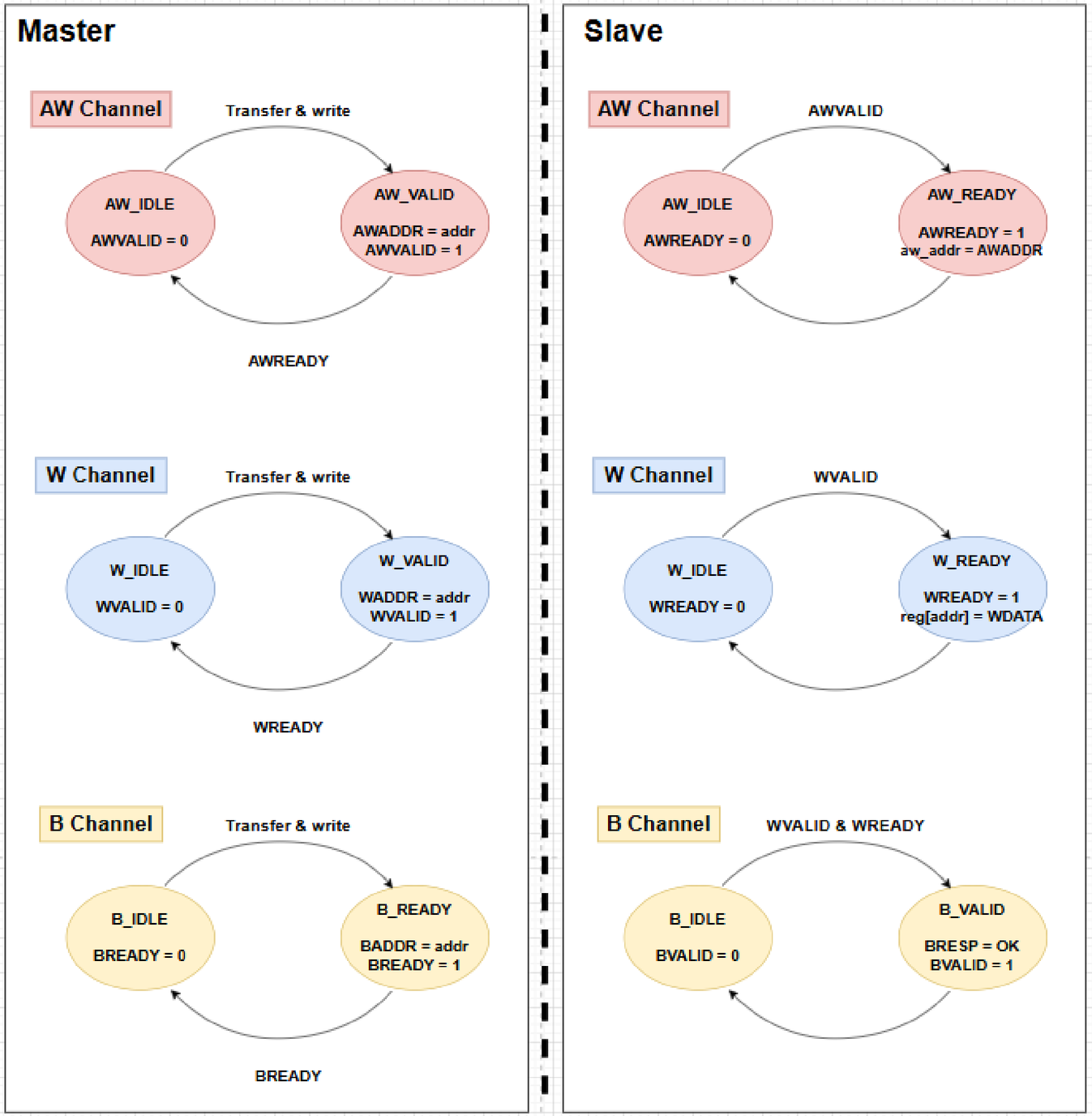


HandShake : Valid == 1 && Ready == 1

handshaking : 송신자 수신자 사이에서 데이터 전송이 안전하게 이뤄졌는지 서로 확인하는 작업

CHAPTER 02

AXI4_Lite

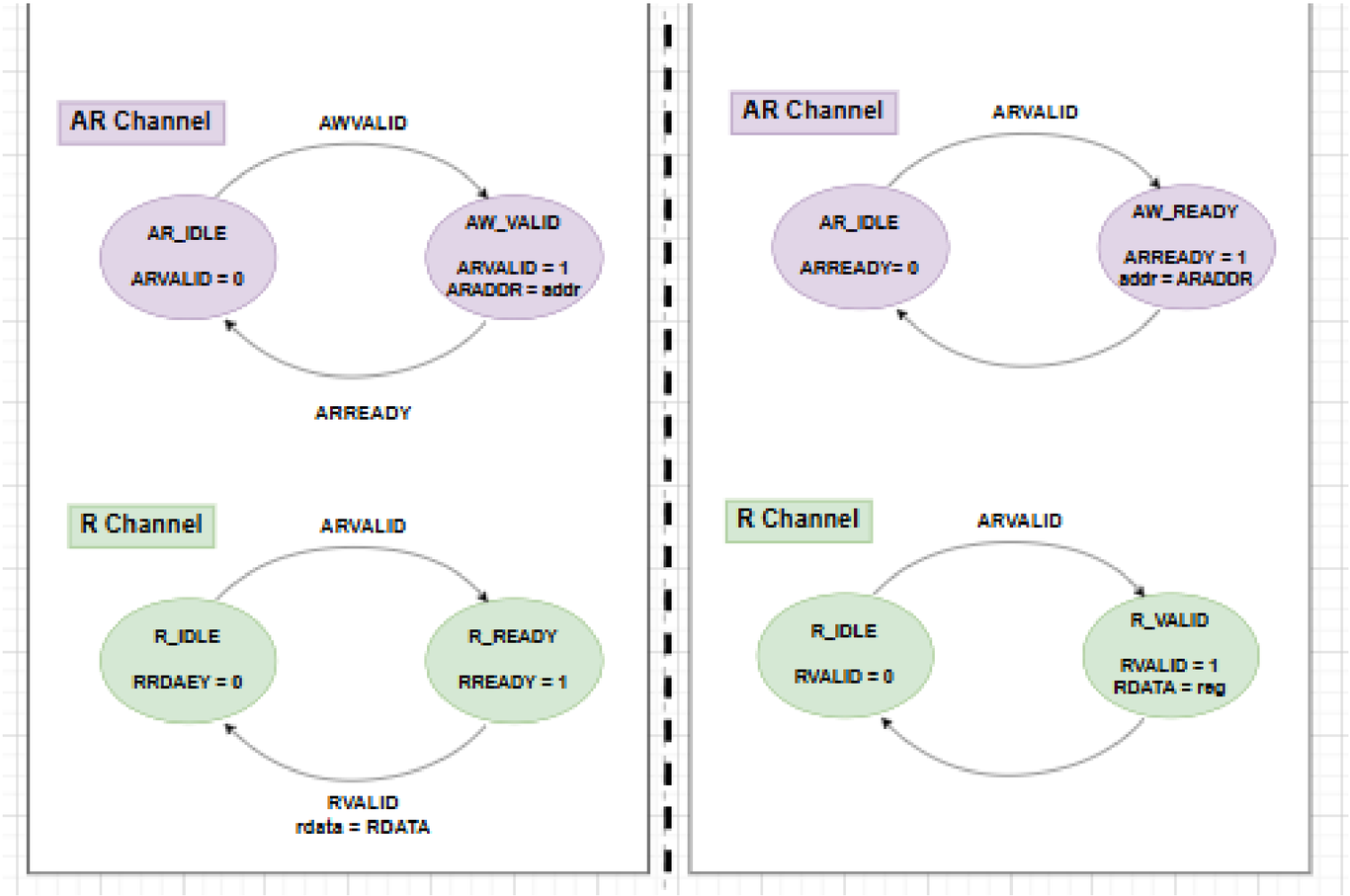


FSM

AXI Protocol에 맞는 FSM

MASTER, SLAVE에 따라 동작

AXI4_Lite



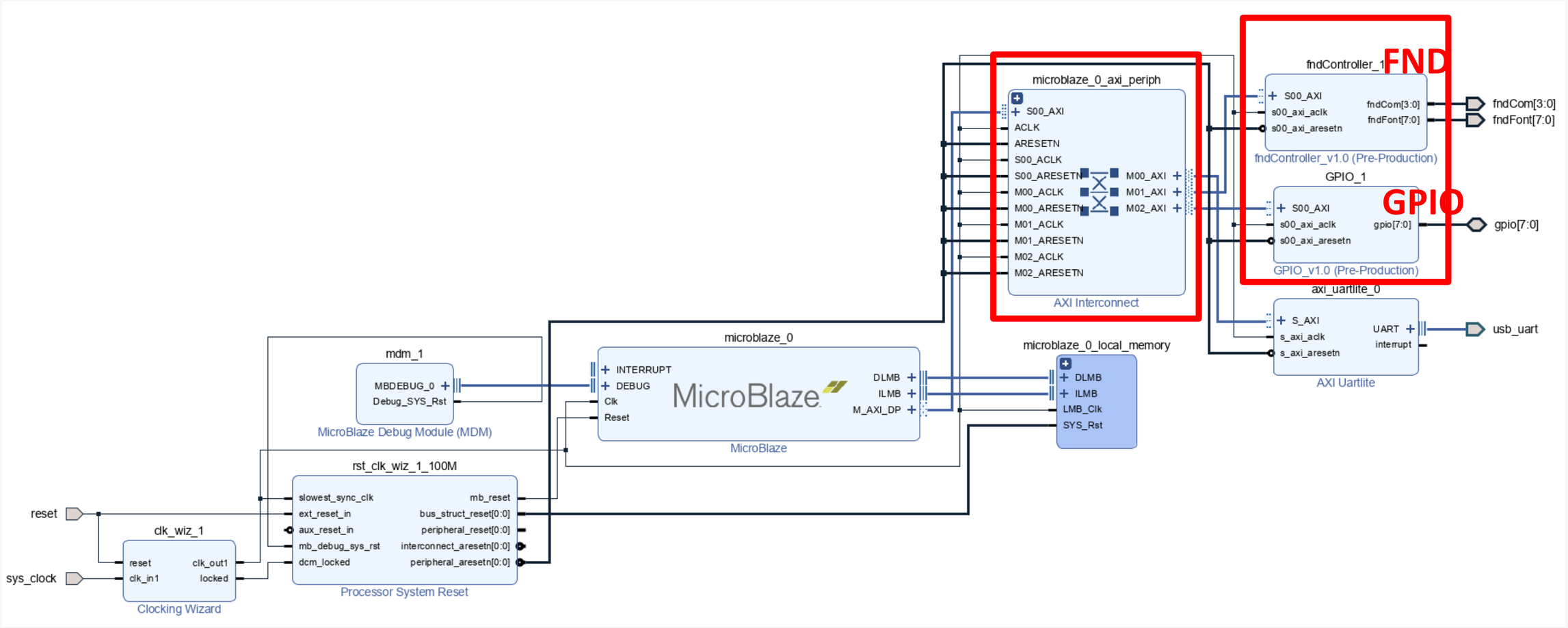
FSM

AXI Protocol에 맞는 FSM

MASTER, SLAVE에 따라 동작

CHAPTER 02

AXI4_Lite



04

Simulation & Verification

CHAPTER 03

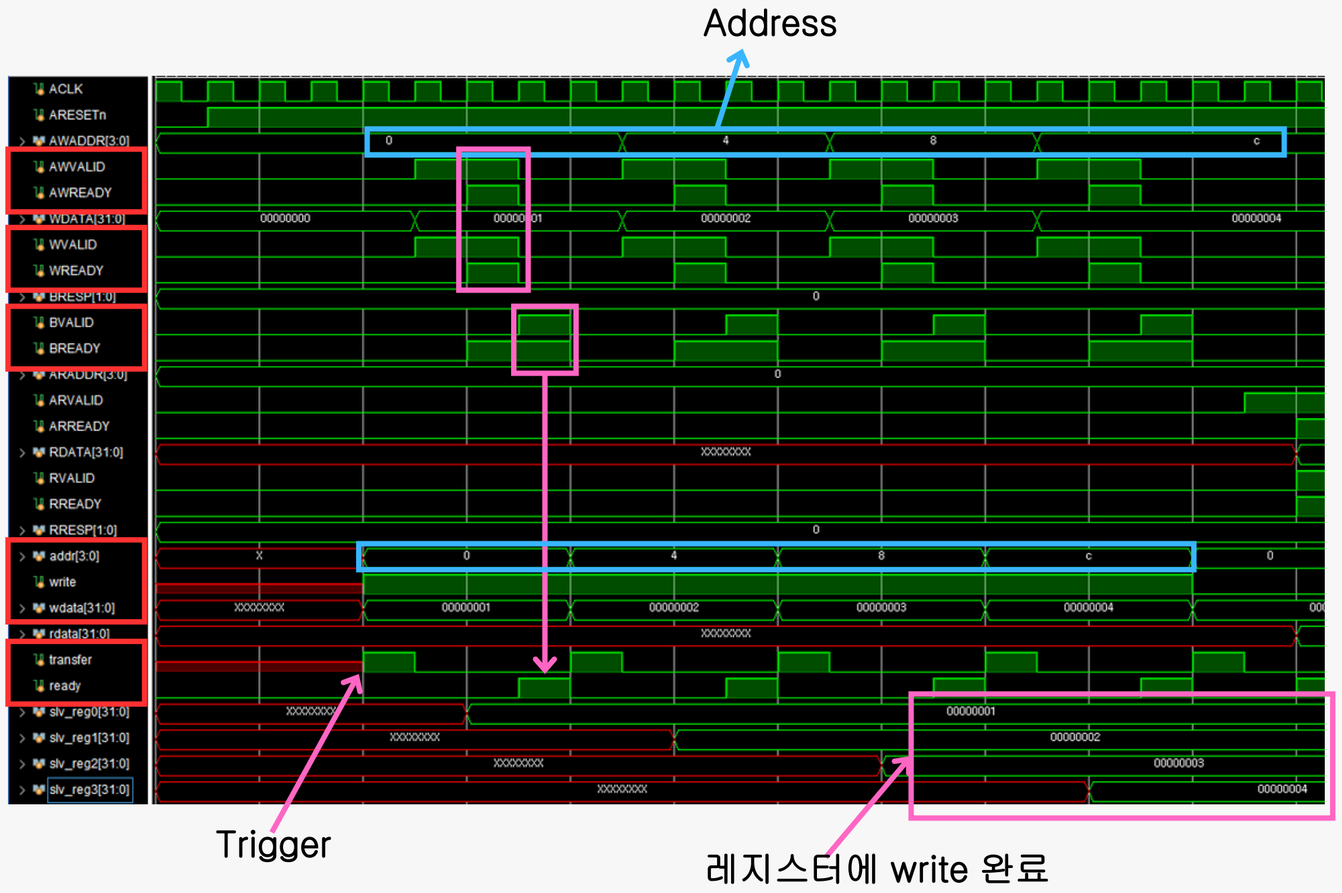
Simulation

```
task axi_write (input logic [3:0] axi_addr, input logic [31:0] axi_wdata);
    @(posedge ACLK);
    addr = axi_addr;
    write = 1;
    wdata = axi_wdata;
    transfer = 1;
    @(posedge ACLK);
    transfer = 0;
    wait(ready);
endtask //axi_write

task axi_read (input logic [3:0] axi_addr);
    @(posedge ACLK);
    addr = axi_addr;
    write = 1'b0;
    wdata = 0;
    transfer = 1;
    @(posedge ACLK);
    transfer = 0;
    wait(ready);
endtask //axi_read

initial begin
    repeat(3) @(posedge ACLK);
    axi_write(4'h0,1);
    axi_write(4'h4,2);
    axi_write(4'h8,3);
    axi_write(4'hc,4);

    axi_read(4'h0);
    axi_read(4'h4);
    axi_read(4'h8);
    axi_read(4'hc);
    #20;
    $finish;
end
```



CHAPTER 03

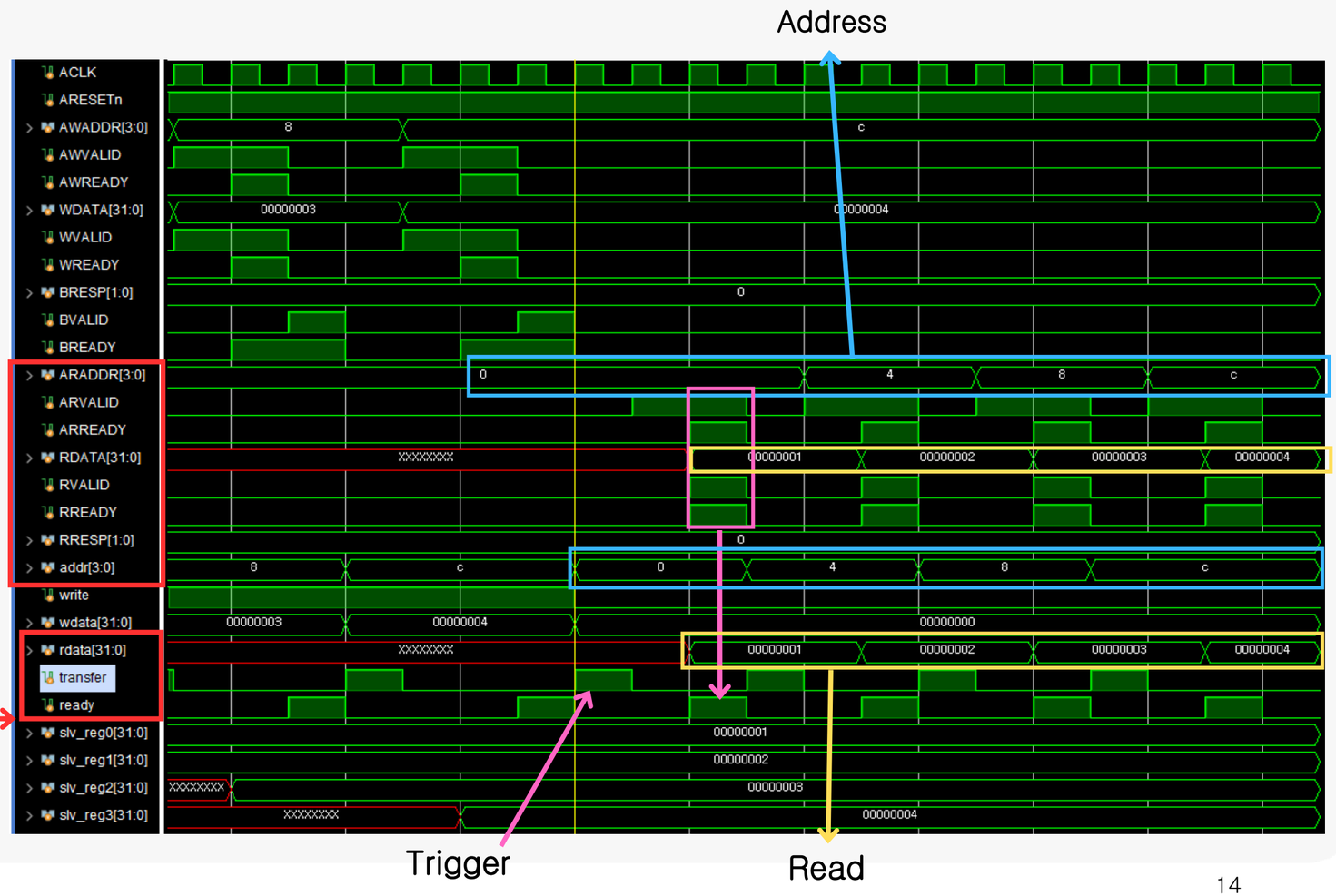
Simulation

```
task axi_write (input logic [3:0] axi_addr, input logic [31:0] axi_wdata);
    @(posedge ACLK);
    addr = axi_addr;
    write = 1;
    wdata = axi_wdata;
    transfer = 1;
    @(posedge ACLK);
    transfer = 0;
    wait(ready);
endtask //axi_write

task axi_read (input logic [3:0] axi_addr);
    @(posedge ACLK);
    addr = axi_addr;
    write = 1'b0;
    wdata = 0;
    transfer = 1;
    @(posedge ACLK);
    transfer = 0;
    wait(ready);
endtask //axi_read

initial begin
    repeat(3) @(posedge ACLK);
    axi_write(4'h0,1);
    axi_write(4'h4,2);
    axi_write(4'h8,3);
    axi_write(4'hc,4);

    axi_read(4'h0);
    axi_read(4'h4);
    axi_read(4'h8);
    axi_read(4'hc);
    #20;
    $finish;
end
```



CHAPTER 03

Simulation

```c
#include <stdio.h>
#include <stdint.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "sleep.h"

typedef struct {
    uint32_t CR;
    uint32_t FDR;
} FND_TypeDef;

typedef struct{
    uint32_t CR;
    uint32_t ODR;
    uint32_t IDR;
} GPIO_TypeDef;

#define FND_BASEADDR  XPAR_FNDCONTROLLER_1_S00_AXI_BASEADDR
#define GPIO_BASEADDR XPAR_GPIO_1_S00_AXI_BASEADDR
#define FND         ((FND_TypeDef *)(FND_BASEADDR))
#define GPIO          ((GPIO_TypeDef *)(GPIO_BASEADDR))

void FND_Init(FND_TypeDef * fnd)
{
    fnd->CR = 0x01;
}

void FND_WriteData(FND_TypeDef * fnd, uint8_t data)
{
    fnd->FDR = data;
}

uint8_t FND_ReadData(FND_TypeDef * fnd)
{
    return fnd->FDR;
}

void GPIO_Init(GPIO_TypeDef * gpio, uint8_t data)
{
    gpio->CR = data;
}

void GPIO_WriteData(GPIO_TypeDef *gpio, uint8_t data)
{
    gpio->ODR = data;
}

uint8_t GPIO_ReadData(GPIO_TypeDef *gpio)
{
    return gpio->IDR;
}
```

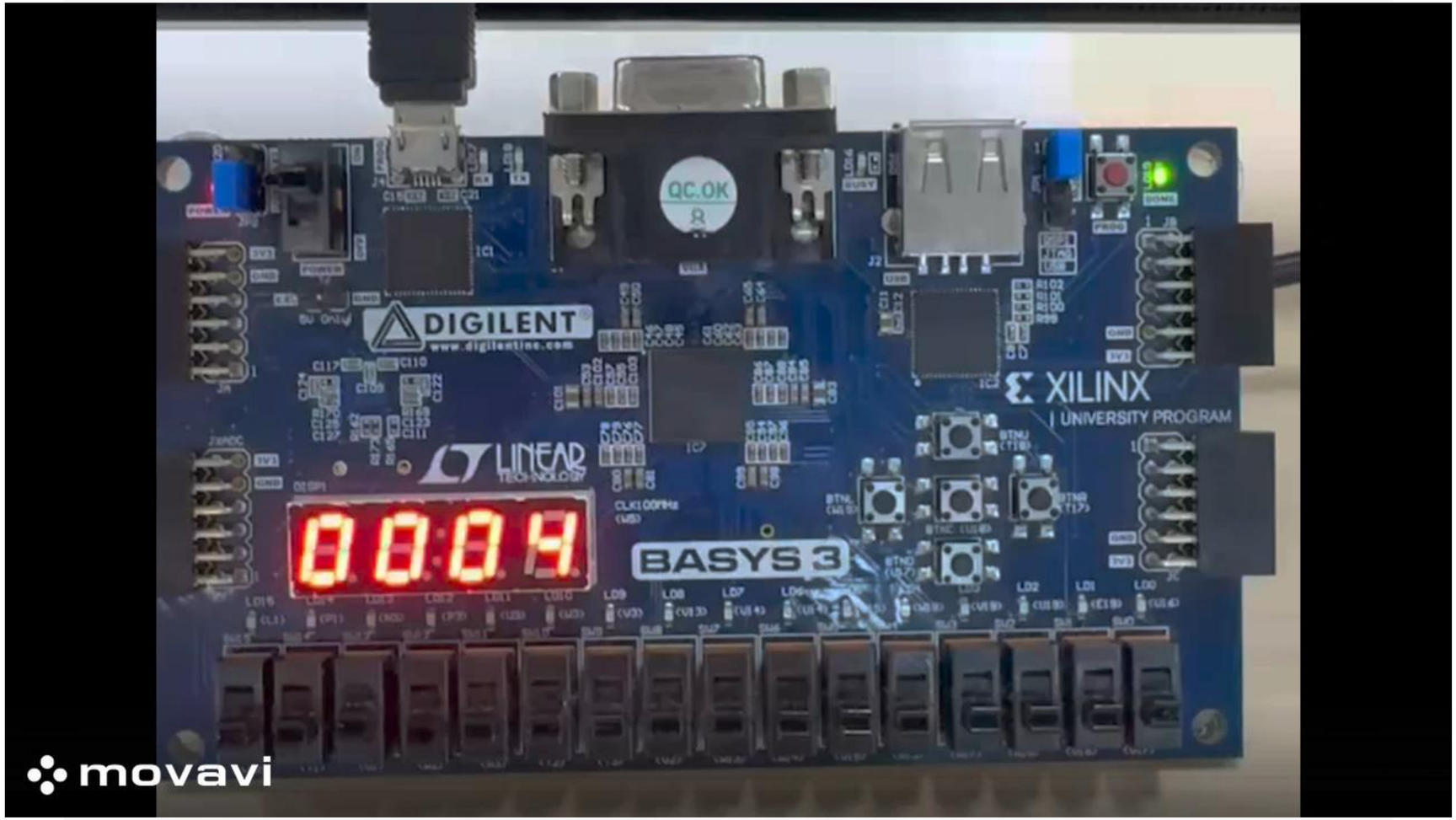
CHAPTER 03

Simulation

```
54  int main()
55  {
56      uint32_t counter = 0;
57      uint8_t led = 0;
58
59      FND_Init(FND);
60      GPIO_Init(GPIO, 0x0f);
61
62      while(1)
63      {
64          FND_WriteData(FND, counter);
65          GPIO_WriteData(GPIO, led);
66          counter++;
67          led = led ^ 0x0f;
68          usleep(100000);
69      }
70
71       return 0;
72  }
```

CHAPTER 03

Simulation



05

Troubleshooting

CHAPTER 02

Troubleshooting

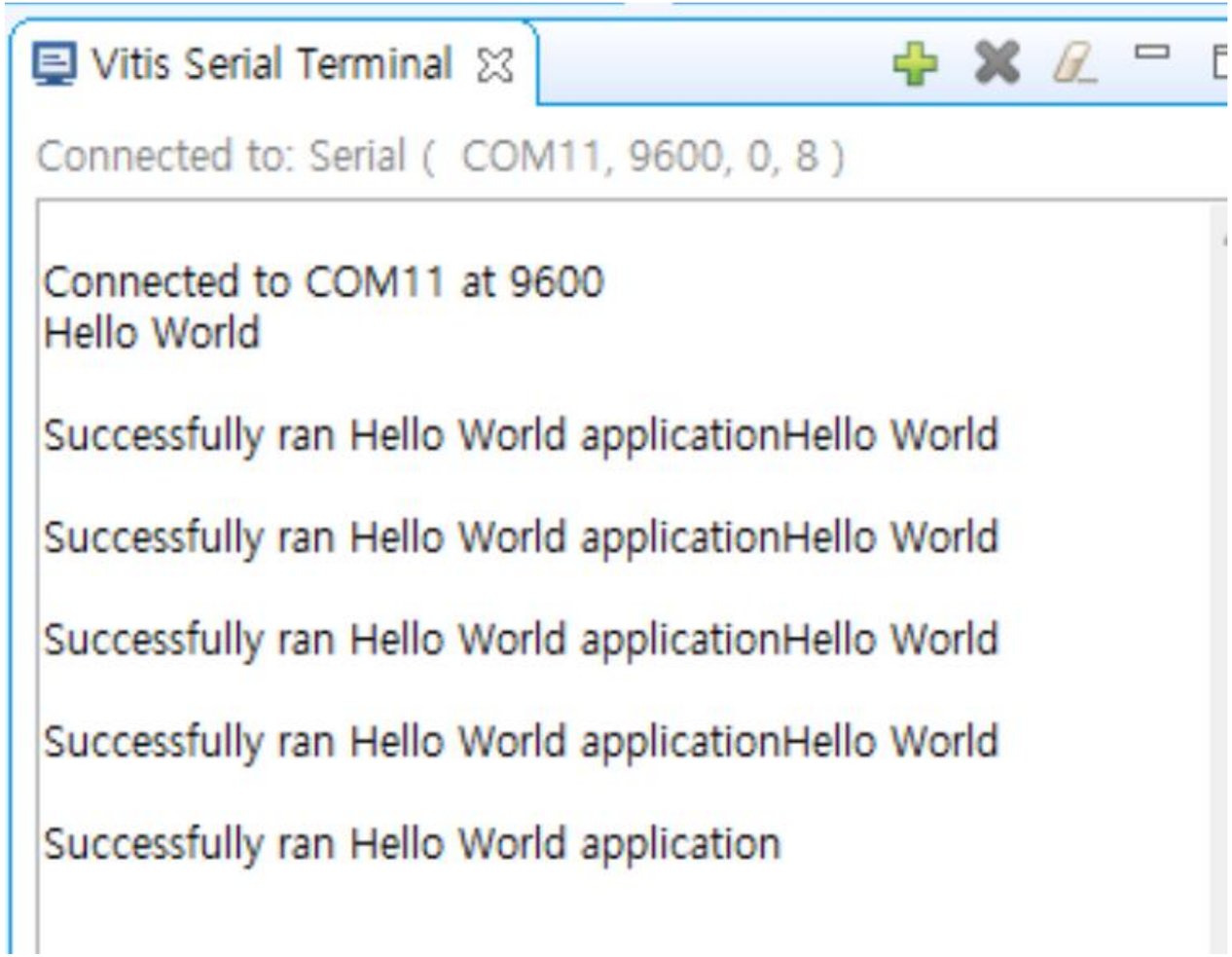
```
11
12    INCLUDEFILES=$(wildcard *.h)
13    LIBSOURCES=$(wildcard *.c)
14    OUTS = $(wildcard *.o)
15
```

MakeFile

Wildcard:
VITIS에서 Printf.h를 사용 못하던 것을 해결

CHAPTER 02

Troubleshooting



MakeFile

해결 후, UART 동작 또한 확인

06

고찰

CHAPTER 06

고찰

01

표준 인터페이스

이전에 진행한 AMBA APB와 차이점을 습득하고, 표준 인터페이스를 사용하며 IP간 통신 구조 단순화

02

규격화

직접 Register Map을 설계하지 않고, 규격 방식으로 쉽게 접근 및 진행하며 다양한 IP 추가 시 인터페이스를 일관성 있게 유지 가능했다.

03

과정의 어려움

APB처럼 직관적이지 않고 신호의 역할 (VALID, READY)를 이해하고, READ/Write 채널이 독립적이라 타이밍을 고려하는 과정이 어려웠다.
이해를 바탕으로 AXI의 장점인 파이프라인을 살려 복잡한 구조를 적용하고 싶다.

감사합니다